

Synthetic biology, simplified

MCSB Bootcamp — Mathematical and Computational Track

Jun Allard

A simplified version of the model from Guido, Collins et al., 2006.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
```

1. Binding and unbinding of a TF to a promoter

```
# parameters
kon = 0.01 # attachment rate, s-1 uM-1
koff = 0.005 # unbinding rate s-1
C = 2.0 # microMolar, concentration of transcription factor

initial_condition = [0, 100]

def dxdt(t, state):
    """db/dt and du/dt for the promoter, given the state [bound, unbound]."""
    b, u = state

    db_dt = +kon * C * u - koff * b
    du_dt = -kon * C * u + koff * b

    return [db_dt, du_dt]

sol = solve_ivp(dxdt, [0.0, 1000], initial_condition)

T = sol.t
```

```

X = sol.y.T # transposed so that X[:, 0] and X[:, 1] read like the Matlab version

fig, ax = plt.subplots()
ax.plot(T, X[:, 0], "-g") # green for bound
ax.plot(T, X[:, 1], "-r") # red for unbound
ax.set_ylabel("Number of promoters in each state")
ax.set_xlabel("Time (seconds)")
plt.show()

```

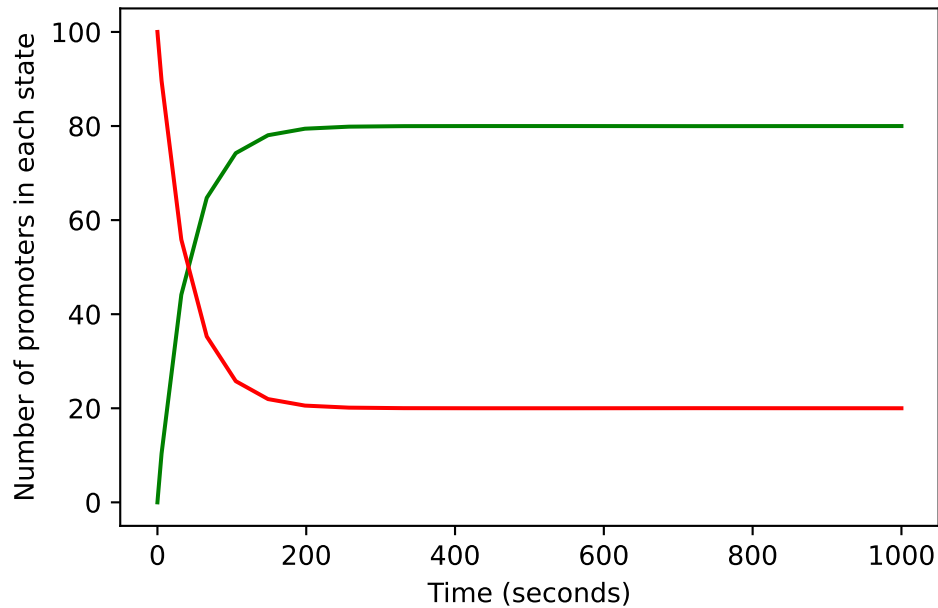


Figure 1: The same two-state model, integrated with an adaptive Runge–Kutta method. Note C has changed.

1.1 Parameter sweep

Run the model once for each of a thousand concentrations of transcription factor, and keep only the last value: the steady-state number of bound promoters.

```

param_array = np.arange(0, 10.001, 0.01)
b_storage = np.zeros(param_array.size)

for i_param, C in enumerate(param_array):

```

```

# Nothing is redefined here.
# The Matlab has to rebuild dbdt, dudt and dxdt on every pass; dxdt above reads C when i

sol = solve_ivp(dxdt, [0.0, 1000], [0, 100])

b_storage[i_param] = sol.y[0, -1]

fig, ax = plt.subplots()
ax.plot(param_array, b_storage, "-b")
ax.set_xlabel("Concentration of transcription factor (uM)")
ax.set_ylabel("Number of bound promoters")
# ax.set_xscale("log")
plt.show()

```

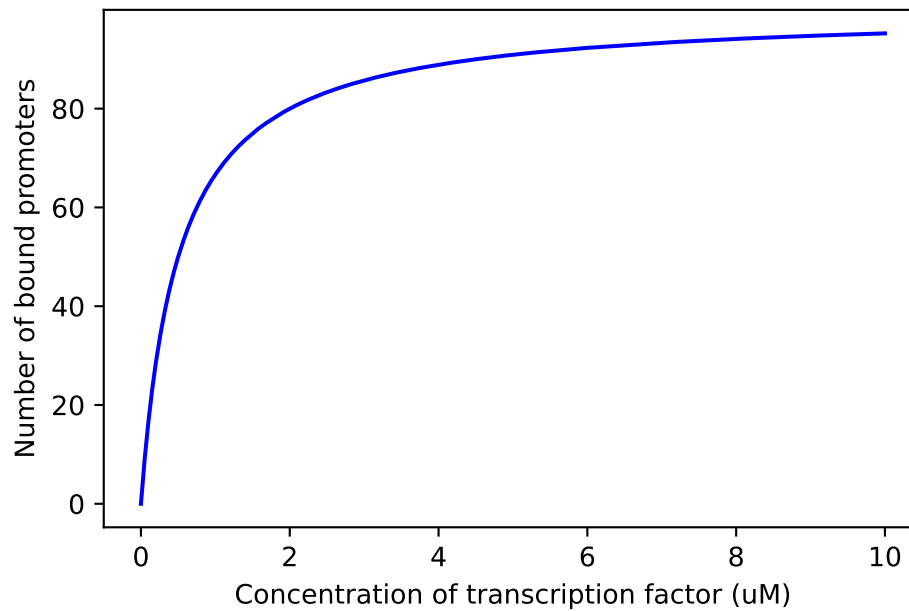


Figure 2: Steady-state bound promoters against transcription factor concentration.

2. mRNA and protein

2.1 Creation and destruction of mRNA and protein

Leave the promoter aside for a moment. mRNA is made at a constant rate and degrades; protein is made in proportion to the mRNA present and degrades.

```

# parameters
gamma_m = 0.2
delta_m = 0.02
gamma_p = 0.04
delta_p = 0.02

initial_condition = [0, 0]

def dxdt(t, state):
    """dm/dt and dp/dt for the mRNA-protein system, given the state [mRNA, protein]."""
    m, p = state

    dm_dt = +gamma_m - delta_m * m
    dp_dt = +gamma_p * m - delta_p * p

    return [dm_dt, dp_dt]

sol = solve_ivp(dxdt, [0.0, 600], initial_condition)

T = sol.t
X = sol.y.T

fig, ax = plt.subplots()
ax.plot(T, X[:, 0], "-r") # red for RNA
ax.plot(T, X[:, 1], "-", color=[0.5, 0, 1]) # purple for protein
ax.set_ylabel("Concentration of RNA (red) and product (purple)")
ax.set_xlabel("Time (seconds)")
plt.show()

```

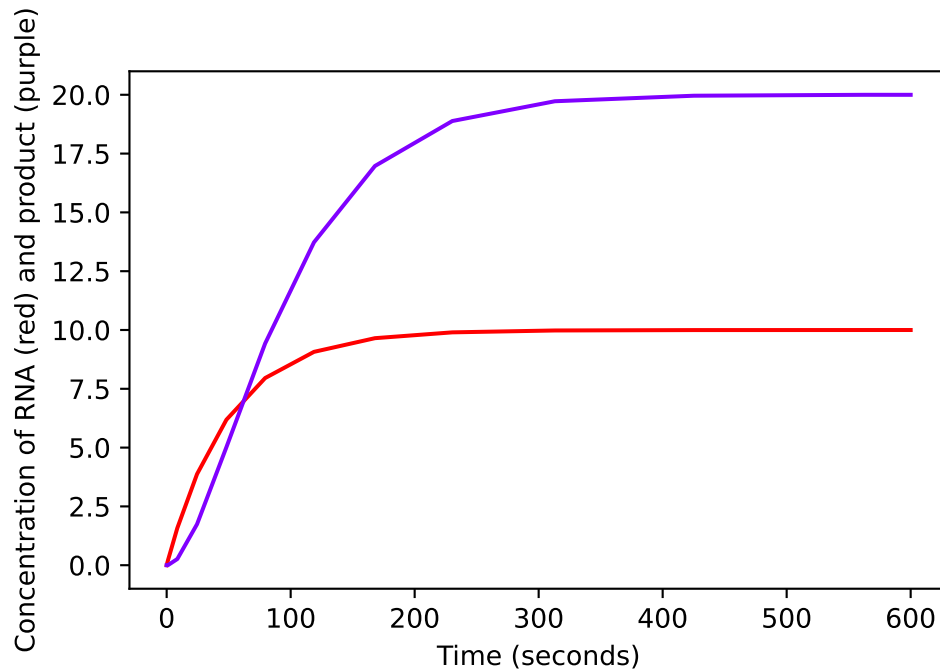


Figure 3: mRNA and protein rising to their steady states, the protein lagging behind.

2.2 Combine the RNA model with promoter binding state

Now put the two halves together. The promoter has four states rather than two — empty, repressor bound, activator bound, both bound — and the rate at which mRNA is produced depends on which state it is in.

```
# parameters
kon = 0.001 # s-1 uM-1
koff = 0.0005 # s-1

delta_m = 0.05
gamma_p = 0.02
delta_p = 0.01

I = 10 # concentration of inhibitory transcription factor (uM)
C = 10 # concentration of activatory transcription factor (uM)

initial_condition = [1, 0, 0, 0, 0, 0]
```

```

def dxdt(t, state):
    """The four promoter states, then mRNA and protein."""
    p0, pr, pa, par, m, p = state

    M = np.array(
        [
            [-kon * C - kon * I, +koff * I, koff, 0],
            [+kon * I, -koff * I - kon * C, 0, +koff],
            [+kon * C, 0, -koff - kon * I, +koff],
            [0, +kon * C, kon * I, -2 * koff],
        ]
    )

    dpromoter_dt = M @ [p0, pr, pa, par]

    # how fast mRNA is made, given which state the promoter is in
    gamma_m = 1.0 * p0 + 0.0 * pr + 2.0 * pa + 1.0 * par

    dm_dt = +gamma_m - delta_m * m
    dp_dt = +gamma_p * m - delta_p * p

    return [*dpromoter_dt, dm_dt, dp_dt]

sol = solve_ivp(dxdt, [0.0, 1000], initial_condition)

T = sol.t
X = sol.y.T

fig, (ax_promoter, ax_product) = plt.subplots(2, 1, figsize=(6, 7))

ax_promoter.plot(T, X[:, 0:4])
ax_promoter.set_ylabel("Number of promoters in each state")
ax_promoter.set_xlabel("Time (seconds)")
ax_promoter.legend(["Empty", "repressor", "activator", "both"])

ax_product.plot(T, X[:, 4], "-r") # red for RNA
ax_product.plot(T, X[:, 5], "-", color=[0.5, 0, 1]) # purple
ax_product.set_ylabel("Concentration of RNA (red) and product (purple)")
ax_product.set_xlabel("Time (seconds)")

fig.tight_layout()

```

```
plt.show()
```

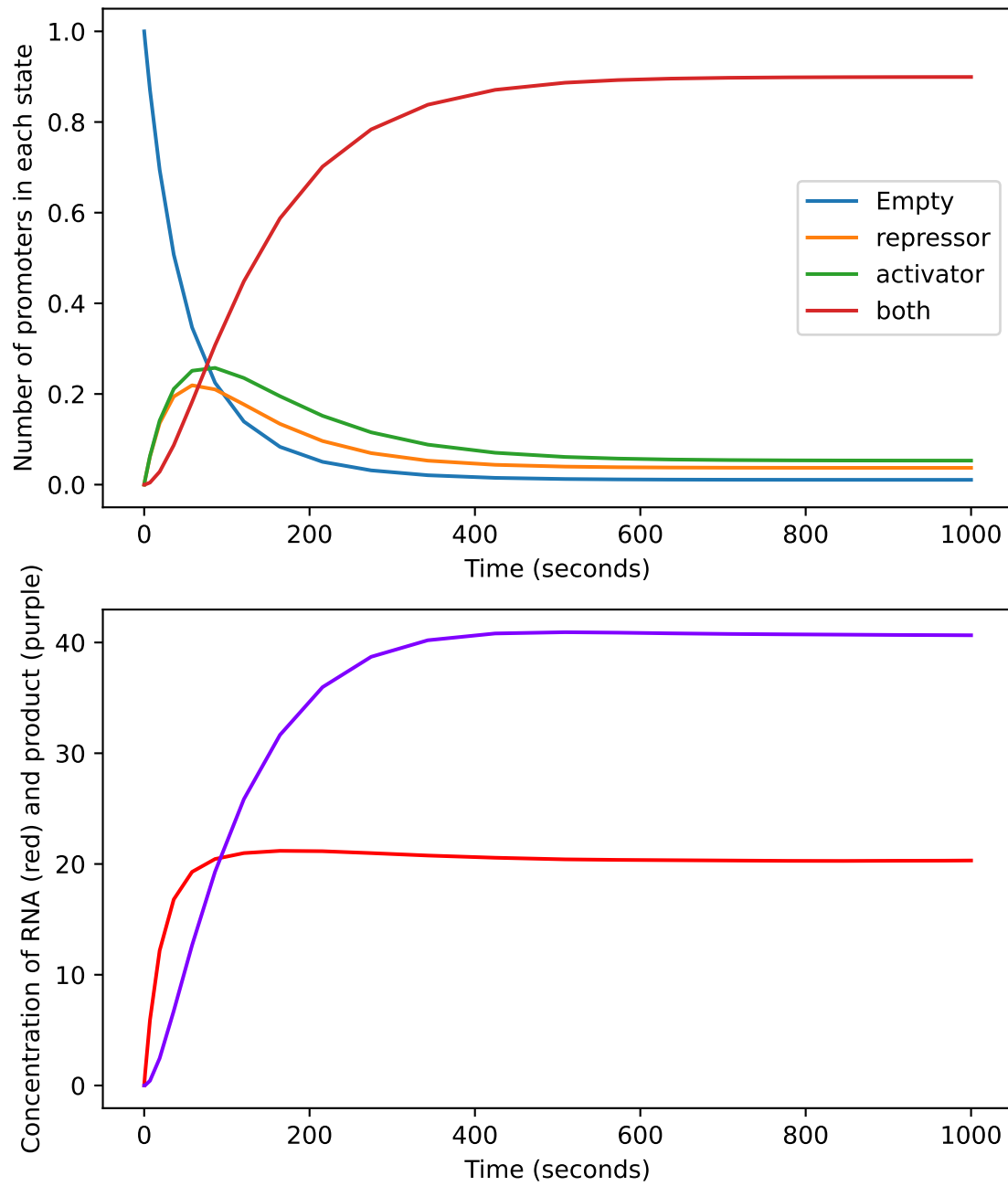


Figure 4: Above, the promoter moving between its four states; below, the mRNA and protein that follow from it.

2.3 Parameter sweep across activator concentration

Same sweep as 1.1, but over the concentration of activating transcription factor, and now reading off the steady-state protein.

```
param_array = np.arange(0, 101, 1)

# param_array = np.logspace(-1, 3, 200) # uncomment to make a log plot

# Named rather than left as a generic g_storage because 2.4 draws it again underneath the feed
g_storage_no_feedback = np.zeros(param_array.size)

I = 10 # concentration of inhibitory transcription factor (uM) # EDIT for HW1

for i_param, C in enumerate(param_array): # concentration of activatory TF (uM) # EDIT for HW1

    sol = solve_ivp(dxdt, [0.0, 100], [1, 0, 0, 0, 0, 0])

    g_storage_no_feedback[i_param] = sol.y[5, -1]

fig, ax = plt.subplots()
ax.plot(param_array, g_storage_no_feedback, "-b")
ax.set_xlabel("Concentration of activating TF (uM)") # EDIT for HW1
ax.set_ylabel("Concentration of product (uM)")
ax.set_ylim(0, 40)
# ax.set_xscale("log") # uncomment to make a log plot
plt.show()
```

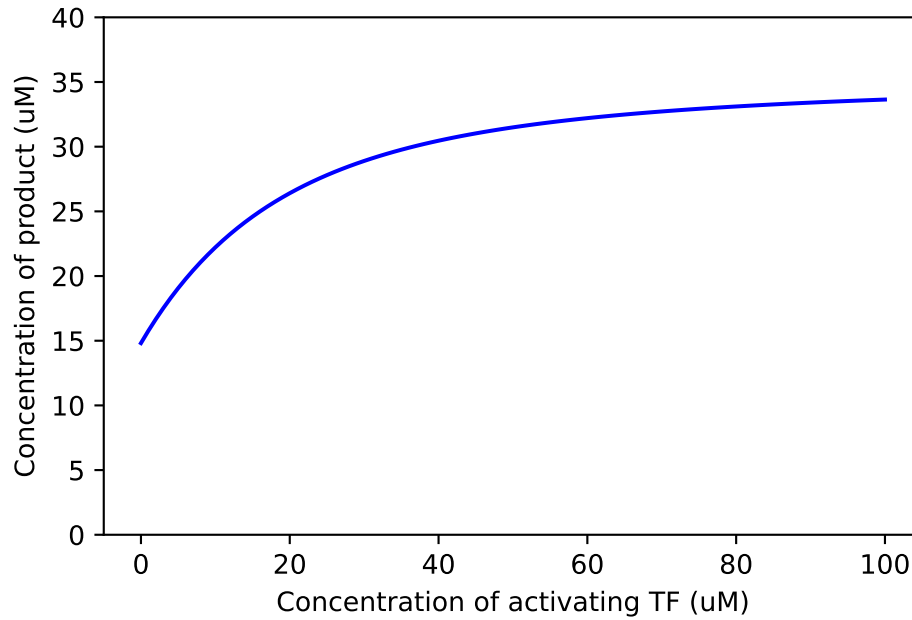



Figure 5: Steady-state product against activator concentration, with the inhibitor held at 10 uM.

2.4 Positive feedback

Add a second gene on the same promoter whose protein is itself an activator of that promoter. The state grows from six variables to eight.

The curve from 2.3 is drawn underneath in pale grey, so the effect of the feedback can be read off one pair of axes rather than by flipping between two figures. Both sweeps have to run over the same `param_array` for that to line up — so if you uncomment the `logspace` line to make a log plot, uncomment it in 2.3 as well.

```
param_array = np.arange(0, 101, 1)

# param_array = np.logspace(-1, 3, 200) # uncomment to make a log plot

g_storage = np.zeros(param_array.size)

# parameters
kon = 0.001 # s-1 uM-1
koff = 0.0005 # s-1
```

```

delta_m = 0.05
gamma_p = 0.02
delta_p = 0.01

gamma_p2 = 0.1

I = 10

def dxdt(t, state):
    """The four promoter states, then two mRNA-protein pairs, the second feeding back on the
    p0, pr, pa, par, m, p, mb, pb = state

    M = np.array(
        [
            [-kon * (C + pb) - kon * I, +koff * I, koff, 0],
            [+kon * I, -koff * I - kon * (C + pb), 0, +koff],
            [+kon * (C + pb), 0, -koff - kon * I, +koff],
            [0, kon * (C + pb), kon * I, -2 * koff],
        ]
    )

    dpromoter_dt = M @ [p0, pr, pa, par]

    gamma_m = 1.0 * p0 + 0.0 * pr + 2.0 * pa + 1.0 * par

    dm_dt = +gamma_m - delta_m * m
    dp_dt = +gamma_p * m - delta_p * p

    dmb_dt = +gamma_m - delta_m * mb
    dpb_dt = +gamma_p2 * mb - delta_p * pb

    return [*dpromoter_dt, dm_dt, dp_dt, dmb_dt, dpb_dt]

for i_param, C in enumerate(param_array): # external activatory transcription factor

    sol = solve_ivp(dxdt, [0.0, 100], [1, 0, 0, 0, 0, 0, 0, 0])

    g_storage[i_param] = sol.y[5, -1]

fig, ax = plt.subplots()

```

```

# 2.3's curve first, so the feedback one is drawn on top of it rather than behind it.
ax.plot(param_array, g_storage_no_feedback, "--", color="0.7", label="No feedback (2.3)")
ax.plot(param_array, g_storage, "-b", label="Positive feedback")
ax.set_xlabel("Concentration of activating TF (uM)")
ax.set_ylabel("Concentration of product (uM)")
ax.set_ylim(0, 40)
ax.legend()
# ax.set_xscale("log") # uncomment to make a log plot
plt.show()

```

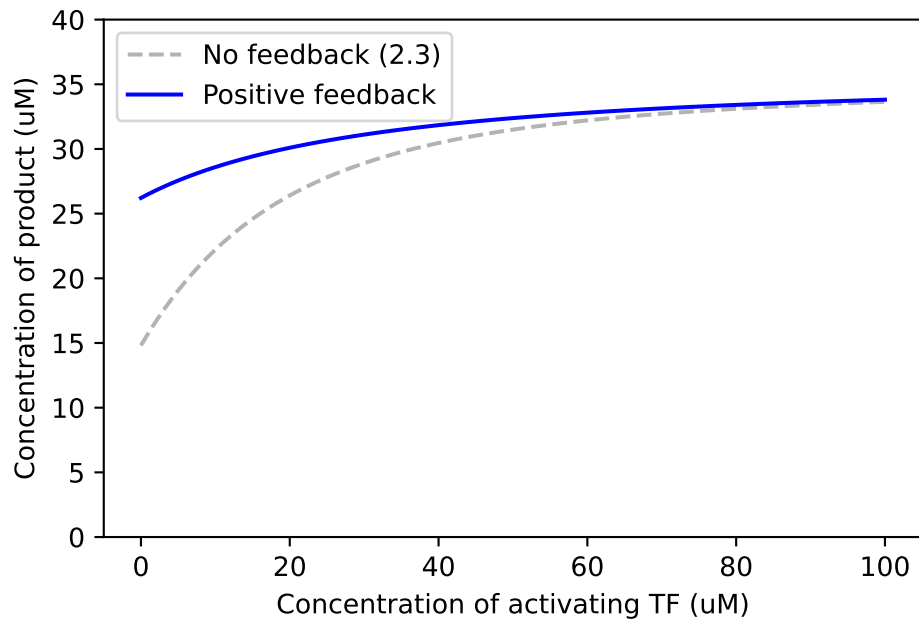


Figure 6: The same sweep with positive feedback added, over the curve from 2.3.